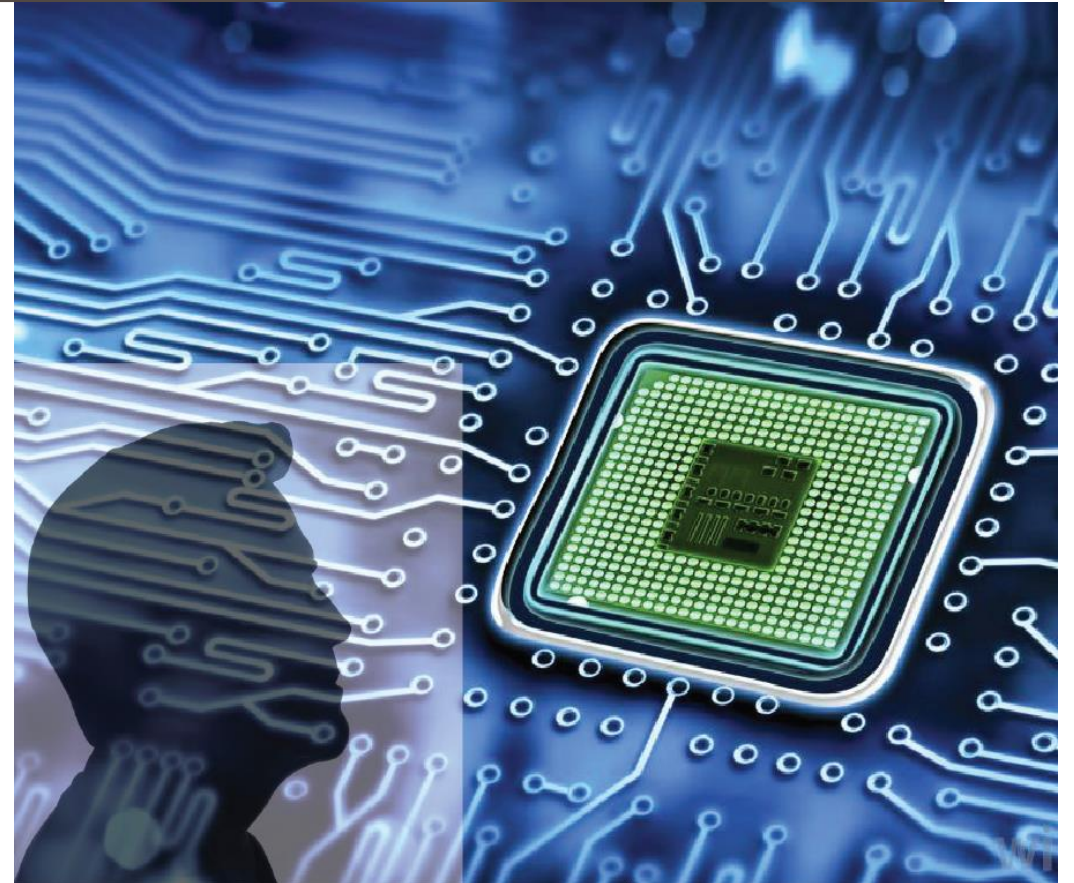


VLSI DESIGN FLOW

Week-4, Lecture-2 Introduction

Sneh Saurabh
4th September, 2024



VDF: Revision/Announcements



Wafer Fabrication and Die Testing

Wafer Fabrication

- Actual fabrication of the design on the silicon wafer is carried out
 - Based on photolithography
 - Consist of hundreds of individual process steps
-
- Fabrication is done layer by layer
 - **Front End Of the Line (FEOL) processes:** fabricate circuit elements such as resistors, capacitors, diodes, and transistors on the lower layers
 - **Back End Of the Line (BEOL) processes:** make interconnections using metallic layers at the top of the wafer



Source: <https://commons.wikimedia.org/wiki/File:Cleanroom1.jpg>
Rudolf Simon, M+W Group GmbH - now Exyte AG, Stuttgart, Germany, CC BY-SA 2.0 DE
<<https://creativecommons.org/licenses/by-sa/2.0/de/deed.en>>, via Wikimedia Commons

Die Testing

- Each die is tested and compared with the expected pattern
- Bad dies are marked and not packaged

Packaging

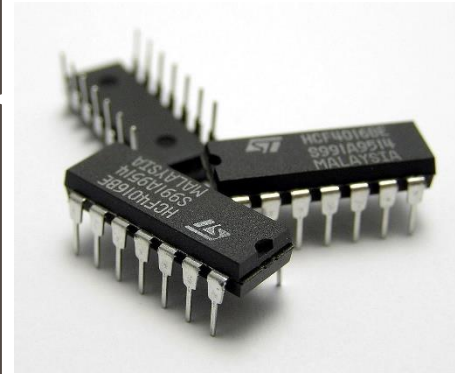
Packaging

- We encapsulate a die in a supporting case known as a package to form a chip.

Functions of a package

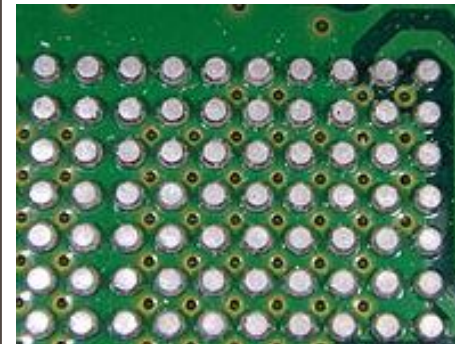
1. Package provides pins for connecting to external environment
 - Characteristics of the package have great impact on the delay of the signal entering/leaving the chip
 - Needs to be carefully designed
2. Package allows dissipation of heat and must be carefully designed
3. Package prevents from mechanical damage and corrosion

Various types and materials are used for package



Dual in-line package (DIP)

Source:
https://commons.wikimedia.org/wiki/File:Three_IC_circuit_chips.JPG



Ball grid array (BGA)

Source:
https://commons.wikimedia.org/wiki/File:Solder_ball_grid.jpg Janke at English Wikipedia, CC BY-SA 3.0
<<https://creativecommons.org/licenses/by-sa/3.0/>>, via Wikimedia Commons

Final Testing and Binning

Final Testing

- Final testing is done to check if packaging is fine
- Burn-in testing: infant mortality
- Quality assurance

Finished/Tested chips

- Can be sent to market directly for sale
- Can be integrated with other chips to form a product and then sent to market for sale

Binning

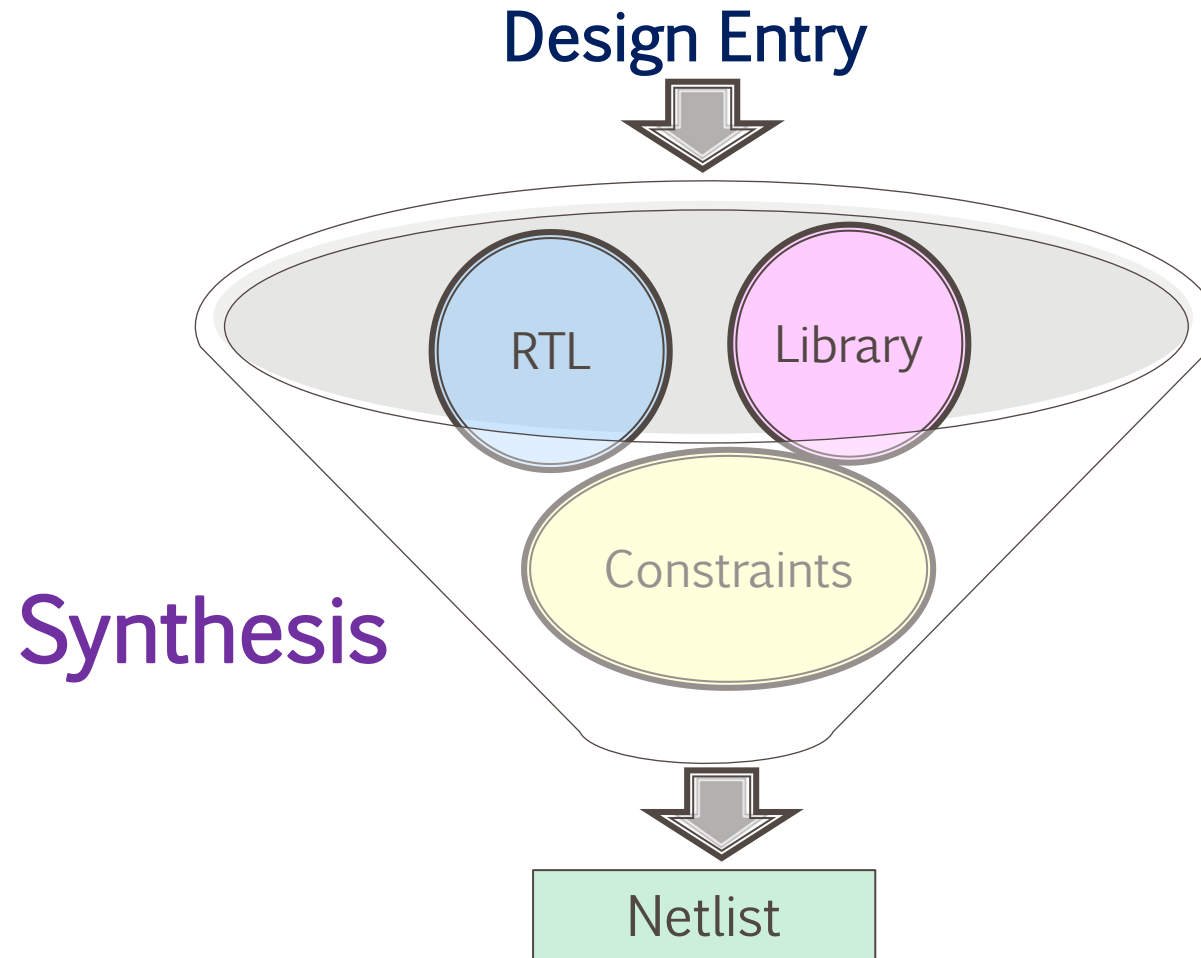
- Classification based on characteristics such as maximum frequency and power dissipation
- Statistical variations in performance
- On-chip delay measurement circuitry
- Assign performance-based price points to different bins.

VDF: Course Content

Week No.	Topic
1	Introduction: Basic Concepts; Design Styles; Designing vs. Fabrication; Processes involved in taking "Idea" to RTL: Software/Hardware Partitioning, Behavioral Synthesis
2	Overview of "RTL to GDS" Design Implementation, Verification and Test Flow; Processes after GDS Tapeout to Final Chip;
3	Design capture: HDL (Verilog), RTL Simulation and Synthesis;
4	Introduction to Formal Methods: Equivalence Checking, BDD, SAT;
5	Library and Constraints; Static Timing Analysis; Power Analysis;
6	Logic Synthesis: Optimization, Technology Mapping;
7	Design for Test (DFT): Fault Models, Scan Insertion,
8	ATPG, BIST
9	Basics of Physical Design
10	Partitioning, Floorplanning, Power Planning;
11	Placement: Techniques and Optimization; Clock Tree Synthesis: Distribution, Skew Optimization;
12	Routing: Global and Detailed; Physical Verification: Extraction, LVS, ERC, DRC;
13	ECO, Sign-off

Logic Synthesis (recap...)

Logic Synthesis: Inputs and Outputs



- RTL: Verilog
 - .v files
- Library : Synopsys Liberty Format
 - .lib file
- Constraints: Synopsys Design Constraints (SDC)
 - .sdc file

Hardware Description Languages (HDL)

HDL: Distinct Features (1)

- Several features that are similar to programming languages such as C, C++
- Additional features in HDL to model hardware easily and realistically

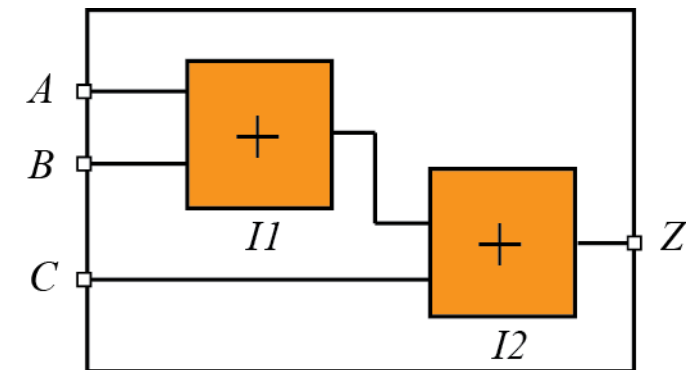
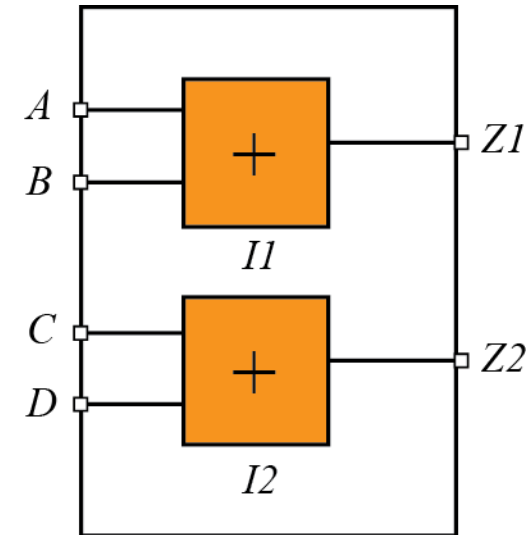
Concurrency

- Computation can be done in parallel in hardware
- HDL must support syntax/semantics to distinguish parallel and sequential operations

Notion of Time

- Describe behavior of circuit with respect to time
- Concurrent/sequential operations
- Ability to create waveform
- Representation of Finite State Machines

Representation of Periodic Signal



HDL: Distinct Features (2)

Electrical Characteristics

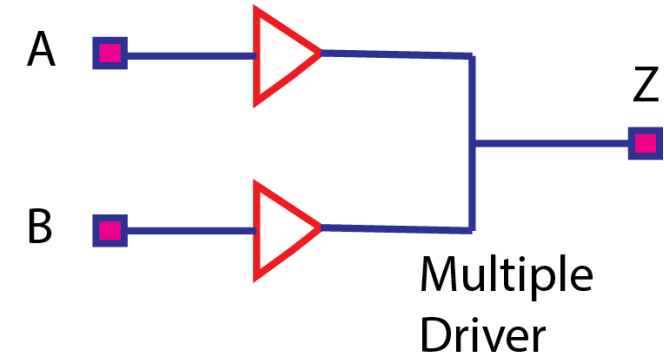
- Tristate
- Driver Strength

Behaviour at clock edges

- Rise/fall edge

Bit-true data types

- Behaviour of buses and how individual bits behave
- How much hardware is required



HDLs : Verilog and VHDL

- Two common HDLs are: Verilog and VHDL

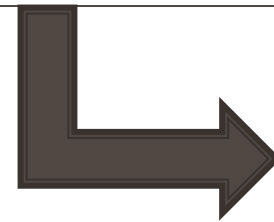
Verilog:

- created in 1983-84 at Gateway Design Automation
- **Verilog** = **Ver**ification + **Log**ic
- Started for the purpose of verification using simulation (fast simulation using Verilog XL)
- Logic synthesis using Design Compiler
- Gained popularity due to simplicity (similarity with C)
- IEEE standardized in 1995 and 2001
- SystemVerilog: superset of Verilog with added functionality for design verification (standardized in 2009)

VHDL: VHSIC Hardware Description Language

- VHSIC: Very High Speed Integrated Circuit
- Initially started as documenting language for integrated circuits in early 1980s
- Verbose and strict type checking
- IEEE standardized in 1987 and 2019

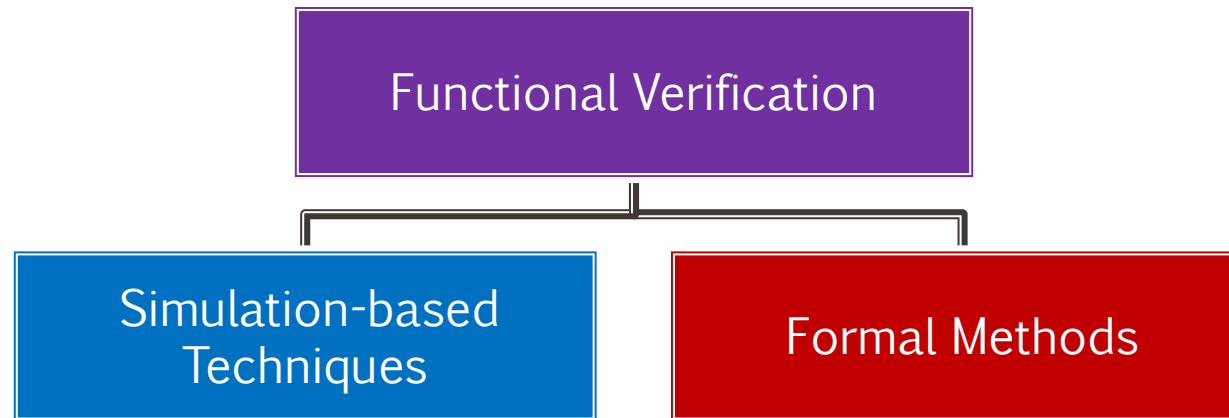
Functional Verification

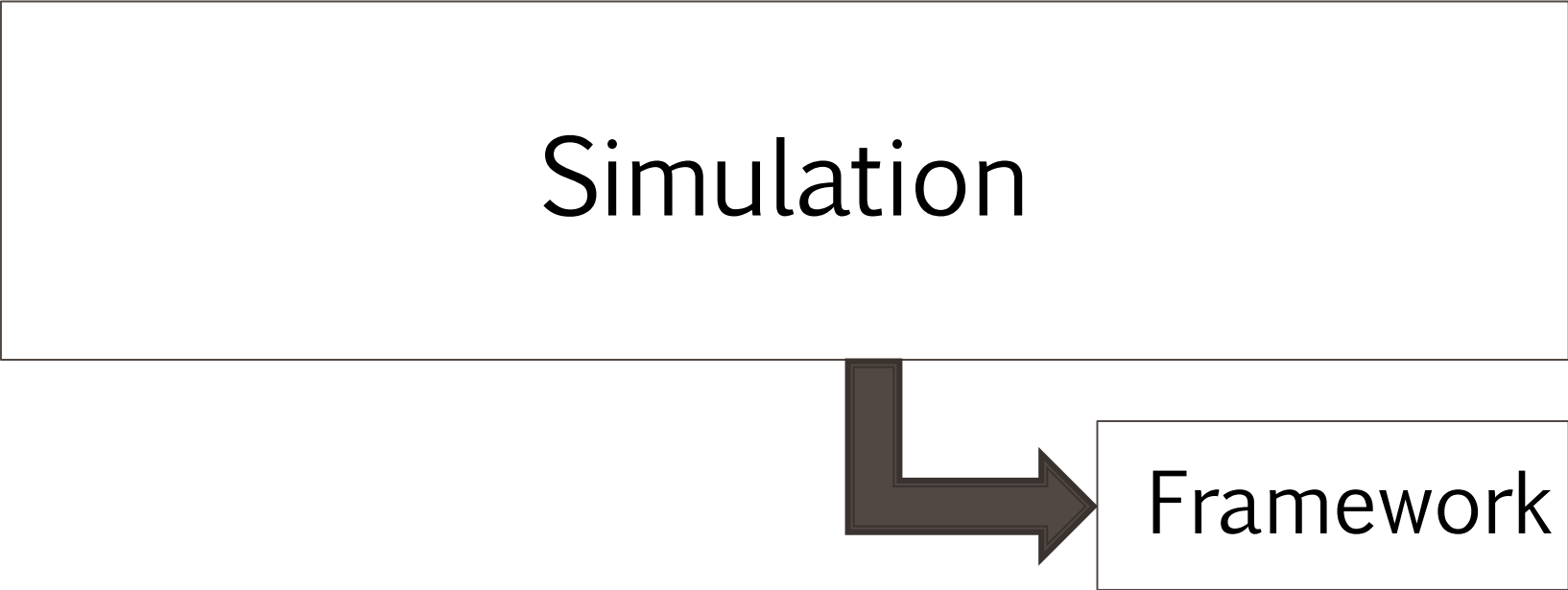


Basics

Functional Verification: Basics

Functional Verification: ensures that the logical functionality of the HDL model matches the given specification





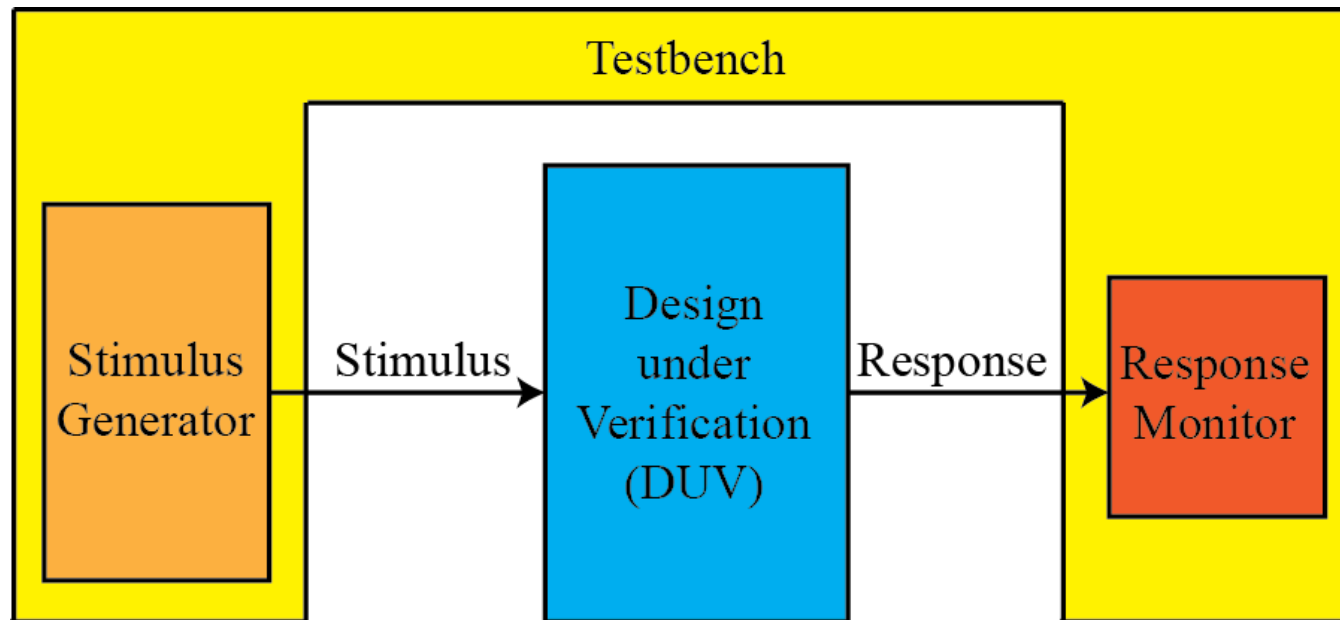
```
graph TD; Simulation[Simulation] --> Framework[Framework]
```

Simulation

Framework

Simulation: Framework

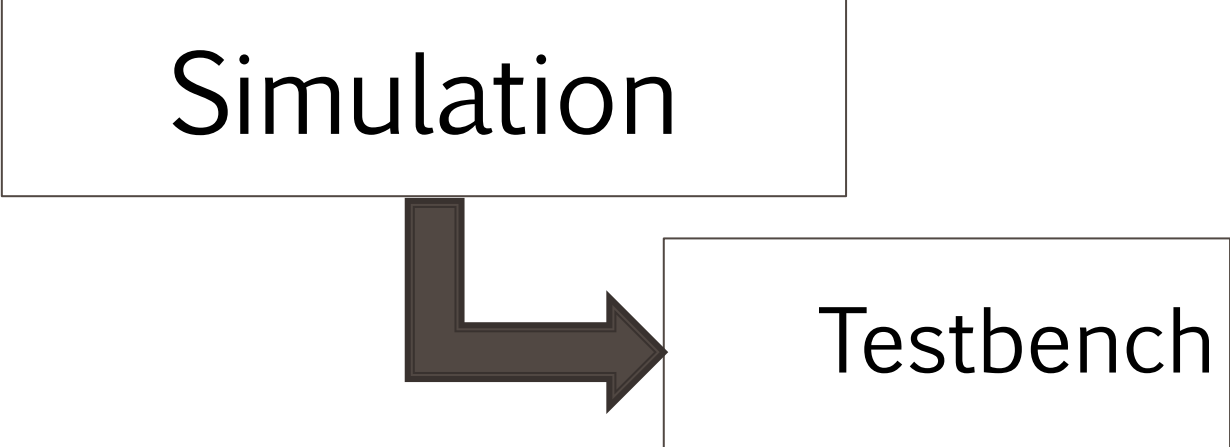
Simulation is performed to verify that the output generated by the design is in agreement with the desired functionality for the given test stimuli



Testbench

- Interacts with simulator
- Applies stimulus
- Observes response

Robustness of verification strongly depends on the testbench



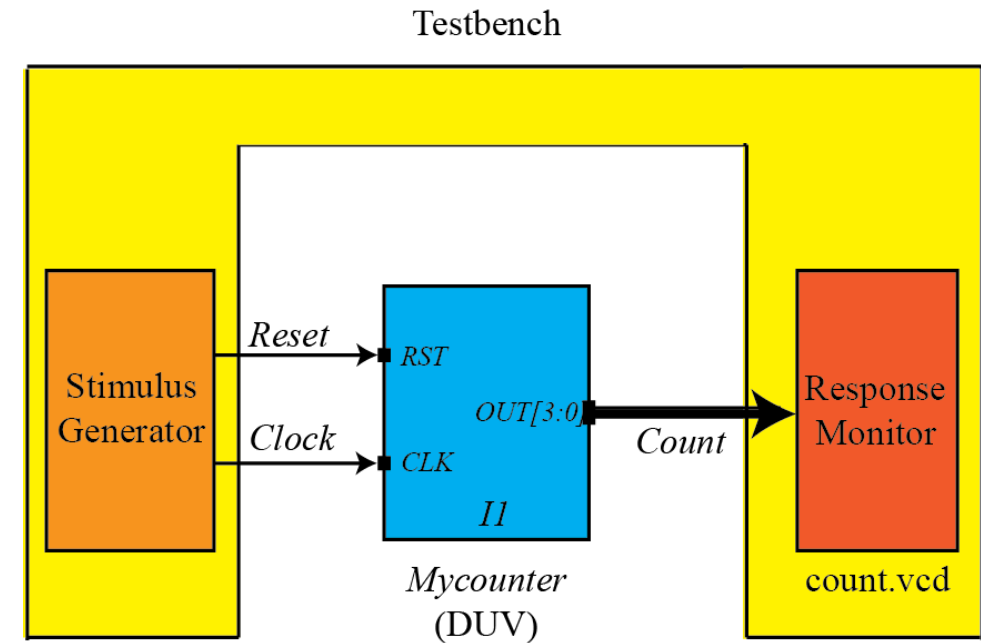
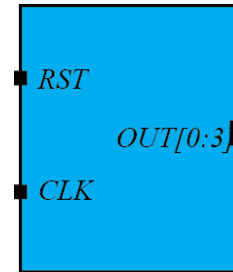
```
graph TD; Simulation[Simulation] --> Testbench[Testbench]
```

Simulation

Testbench

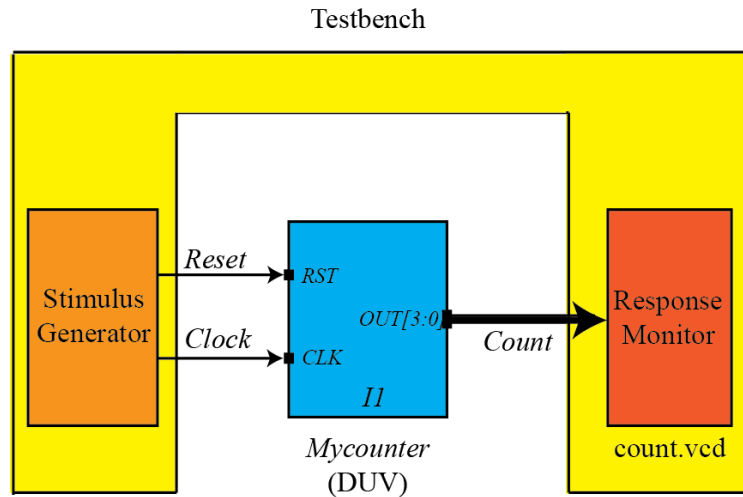
Testbench: Illustration (1)

```
module Mycounter(CLK, RST, OUT);  
  
  input CLK, RST;  
  output [3:0]OUT;  
  reg [3:0]OUT;  
  
  always @ (posedge CLK)  
  begin  
    if (RST == 1'b1)  
      OUT <= 4'b0000;  
    else  
      OUT <= OUT + 1;  
    end  
  end  
endmodule
```



- Instantiate the DUT
- Generate test signals
- Monitor the signals and save it in a file

Testbench: Illustration (2)



- Instantiate the DUT
- Generate test signals
- Monitor the signals and save it in a file

```
module Testbench();
    reg Clock, Reset;
    wire [3:0] Count;

    // instantiate the DUT and make connections
    Mycounter I1(.CLK(Clock), .RST(Reset), .OUT(Count));
    // initialize the testbench
    initial begin
        Clock = 1'b0; Reset = 1'b1; // reset the counter at t=0
        #100 Reset = 1'b0; // remove reset at t=100
        #2000 $finish; // end the simulation after t=2000
    end
    // generate stimulus (in this case clock signal)
    always #50 Clock = ~Clock; // clock period = 100
    // monitor the response and save it in a file
    initial begin
        $dumpfile("count.vcd"); // specifies the VCD file
        $dumpvars; // dumps all variables
        $monitor("%d,%b,%b,%d", $time, Clock, Reset, Count);
    end
endmodule
```

Code Coverage

A good testbench:

- Performs comprehensive design verification without wasting resources on dispensable tasks
- Contains a sufficiently diversified set of stimuli
 - Adding more test stimuli for already tested features may be redundant

Code Coverage:

- Identifies sections of DUV's source code that did not execute during verification.
- Monitor the progress of verification effort

- **Line coverage:** number of times each RTL statement is executed during simulation
- **Branch coverage:** whether all branches of the code (as in if-else and case statements) were exercised in simulation

- **State coverage:** whether all the states of an FSM have been activated and all the state transitions traversed.
- **Toggle coverage:** whether all variables or bits in variables have risen and fallen.

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.